Assignment 1

Course	**Artificial Intelligence**
Instructor	**Ms. Rushda Muneer**
Section	**BCS-6B, BCS-6C & BCS-6G**
Marks	**100**

Important Guidelines:

1. The assignment is programming assignment.
2. You must submit the soft copy (RubiksCube2x2x2.py extension) on google classroom.
3. The file must consist of Your Roll Number Section enclosed in the comments at the top.
4. You are not allowed to copy assignment from each other
5. There will be no extension for deadline (12th March, 2025 11:59pm)
6. No late submissions are allowed

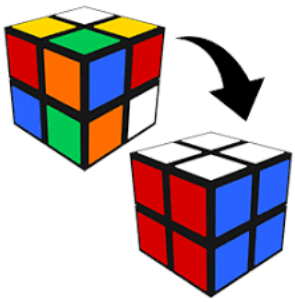
A* Search: Rubik's Cube

For this assignment, you will implement **A* Search**, use it to solve Rubik's Cube.

Rubik's Cube is a classic 3-Dimensional combination puzzle created in 1974 by inventor Erno Rubik. The standard version consists of a 3×3×3 cube. See Online Rubik's Cube Simulator (https://ruwix.com/online-puzzle-simulators/?d=3) .

The cube looks like this:

There are six faces that can be rotated 90 degrees in either direction. The goal is to rearrange the colors to make each face monochromatic by rotating slices.



One important fact about Rubik's Cube is that it has a large **state space**, with approximately 4.3×10^{19} different possible configurations for a 3×3×3 cube. By comparison, the 8-puzzle state space has 362,880 different configurations of the eight tiles and blank space, which is a much smaller number. For this reason and for the sake of practicality, we tackle a 2×2×2 Rubik's Cube in this assignment which has around 3,674,160 possible states, still a large number.

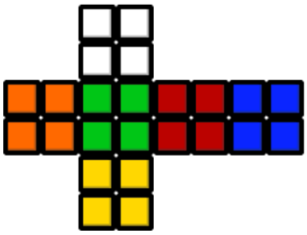
The code for this assignment should be written in Python 3 (.py extension) to run on VS Code.

Implementation Setup

The various parts of this assignment will require a shell script that can pass an argument to your code—thus allowing you to use **python3** while allowing us to be able to run your code with a consistent interface. However, **you may only use built-in standard libraries (e.g., math, random, etc.); you may NOT use any external libraries or packages**.

State Representation

For this assignment, you first need to create a representation for the state of the cube. Let's assume that we can represent the cube as a two-dimensional representation, which we can print like this:



```
   WW
   WW
OO GG RR BB
OO GG RR BB
   YY
   YY
```

Each letter represents a color, and each 2×2 square represents a face in the cube. The goal is for each face to have all its tiles of the same color.

Write a class **Cube** that takes a string representation for a cube with the **proper size**. We will assume that the input string will have the following format:

"WWWW RRRR GGGG YYYY OOOO BBBB"

The indexing order that maps this string to the 2-dimensional representation of the cube is as follows:

```
       0  1
       2  3
16 17  8  9   4  5   20  21
18 19  10 11  6  7   22  23
       12 13
       14 15
```

If the state representation argument is not provided here and in any of the future commands, please use the board above as the default.

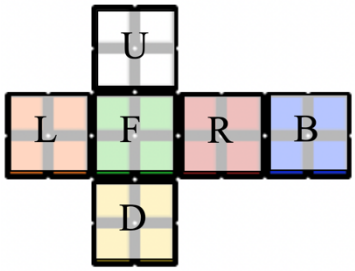
Identifying Solutions

Write a method that determines whether the given cube is at a goal state. This should be very easy: if all the faces have the same color, then you have reached the solution state. Then, augment the possible commands to accept a "**goal**" command that returns "**True**" or "**False**" depending on whether the given board is at the solution state or not.

Move generation

The notation used for Rubik's Cube moves is **F**, **R**, **U**, **B**, **L**, and **D** capital letters. Each of these letters' marks one 90-degree clockwise rotation of one face of the cube – in the

order **F**ront, **R**ight, **U**p, **B**ack, **L**eft, and **D**own. And if the letter is followed by an apostrophe, that means a counterclockwise rotation.



For rule (move) generation, implement a function **'applyMove'** that given a state and a move, performs the move in the state.

Important note: each move described above is actually a permutation of the indices of the colors. For example, after applying **U** the position of the colors will change as below:

[2,0,3,1,20,21,6,7,4,5,10,11,12,13,14,15,8,9,18,19,16,17,22,23]

That is, the colors originally in positions 2,0,3,1 are moved to positions 0,1,2,3; the colors originally in positions 20,21 are moved to positions 4,5; the colors originally in positions 10,11 remain in positions 10,11, etc.

A* Search should be written so that it uses Best-First Search with Algorithm A*, i.e., with a heuristic of the form *f(node) = depth(node) + h(node)*, where *h(node)* does not overestimate the distance to a goal. You will need to develop an admissible heuristic *h(node)*. Use your insight and ingenuity for this.

Program Output

The output should print the following:

- the entire sequence of moves and resulting states to solve the changed cube,
- the number of nodes that were generated,
- the number of nodes that were expanded,
- and the time that the function took to find the solution.